Containerization & Kubernetes Deployment Challenge

Name: Jagdish Parthsarthi Babalsure

PRN: 202301040166

Course: CND (MDM)

GitHub Repository: https://github.com/jagdishbabalsure/CLOUD-NATIVE-DEVOPS-MDM/tree/main/Assignment%202

Introduction

Containerization is a key concept in cloud-native application development. It packages an application and its dependencies into a portable unit that can run consistently across different environments. Kubernetes provides container orchestration capabilities, including deployment management, scaling, service discovery, and recovery from container failures.

This assignment demonstrates the containerization and deployment of the AURA.STUDIO web application using Docker and Kubernetes. The application is served using Nginx and deployed to a local Kubernetes cluster created with Minikube.

Docker is used to build an image containing the application's static files and Nginx configuration. Kubernetes manages the application through a Deployment with three replicas and exposes it through a NodePort Service. The deployment is verified using Kubernetes command-line tools and a web browser.

Objectives

1. Containerize a static web application using Docker.
2. Create a Docker image named nginx-assignment:1.0.
3. Run and test the application in a standalone Docker container.

4. Start a local Kubernetes cluster using Minikube.
5. Deploy the application using a Kubernetes Deployment with three replicas.
6. Configure container resource requests and limits, along with liveness and readiness probes, in the Deployment manifest.
7. Expose the application using a Kubernetes NodePort Service.
8. Verify the deployment, Pods, Service, and application accessibility using kubectl.
9. Maintain the project source code and deployment configuration in GitHub.

Application Overview

The project contains the AURA.STUDIO web application and its deployment configuration.

- **Dashboard (index.html):** Displays assignment-related information and student details.
- **Storefront (store.html):** Provides the e-commerce interface, including product browsing, search and filtering, shopping bag interactions, subtotal calculations, and checkout flow.
- **Stylesheet (style.css):** Defines the application's visual appearance and responsive layout.
- **JavaScript (app.js):** Contains application interaction logic and related functionality.
- **Nginx configuration (nginx.conf):** Configures the web server for serving the application's static content and any additional settings defined in the file.

The application is packaged into a Docker image and deployed on a local Kubernetes cluster.

Technology Stack

Technology	Purpose
HTML5	Structures the web pages
CSS3	Provides styling and responsive layouts
JavaScript (ES6+)	Implements interactive application features
Nginx	Serves static web content
Docker	Builds and runs the container image
Kubernetes	Manages application deployment and replicas
Minikube	Runs a local Kubernetes cluster
kubectl	Manages and inspects Kubernetes resources
PowerShell	Supports deployment automation through deploy.ps1
Git and GitHub	Provide version control and source-code hosting

Project Structure

The project directory contains the following files:

Assignment 2/

├── Dockerfile

├── deploy.ps1

├── index.html

├── store.html

├── app.js

├── style.css

├── hero.jpg

├── nginx.conf

```
├── README.md
└── k8s/
    ├── deployment.yaml
    └── service.yaml
```

File descriptions:

- Dockerfile: Defines the Docker image and the files copied into it.
- deploy.ps1: Contains PowerShell deployment automation.
- index.html and store.html: Contain the dashboard and storefront pages.
- app.js and style.css: Provide JavaScript functionality and styling.
- hero.jpg: Supplies the hero image used by the application.
- nginx.conf: Contains the Nginx server configuration.
- deployment.yaml: Defines the Kubernetes Deployment.
- service.yaml: Defines the Kubernetes Service.
- README.md: Documents the project and its setup instructions.

Dockerfile and Image Creation

The Dockerfile uses nginx:1.25-alpine as the base image. Alpine-based images provide a relatively small base environment suitable for serving static web content.

The Dockerfile removes the default Nginx HTML files, copies the application's HTML pages and supporting assets into the web root, and installs the project's Nginx configuration.

Dockerfile

```
FROM nginx:1.25-alpine


LABEL maintainer="Jagdish Parthsarthi Babalsure"

LABEL description="Containerized Nginx web application for
Kubernetes assignment"

LABEL version="1.0"

RUN rm -rf /usr/share/nginx/html/*

COPY *.html /usr/share/nginx/html/

COPY style.css /usr/share/nginx/html/

COPY app.js /usr/share/nginx/html/

COPY hero.jpg /usr/share/nginx/html/

COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --
retries=3 \

    CMD wget --quiet --tries=1 --spider http://localhost:80/ ||
exit 1

CMD ["nginx", "-g", "daemon off;"]
```

Building the Docker Image

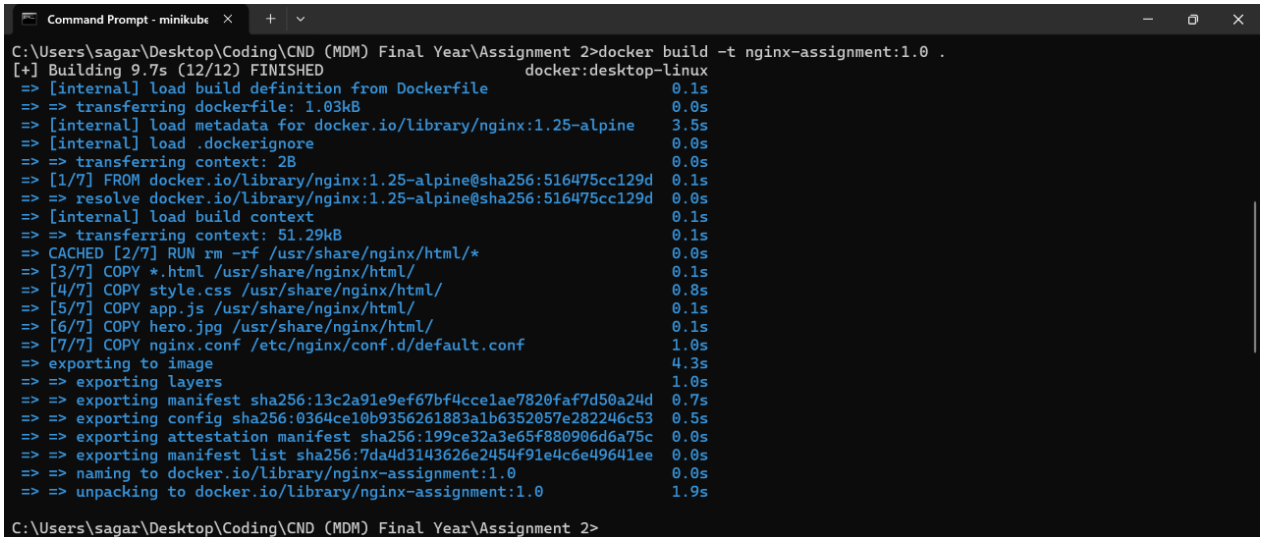
Open Command Prompt or PowerShell in the project root directory and execute:

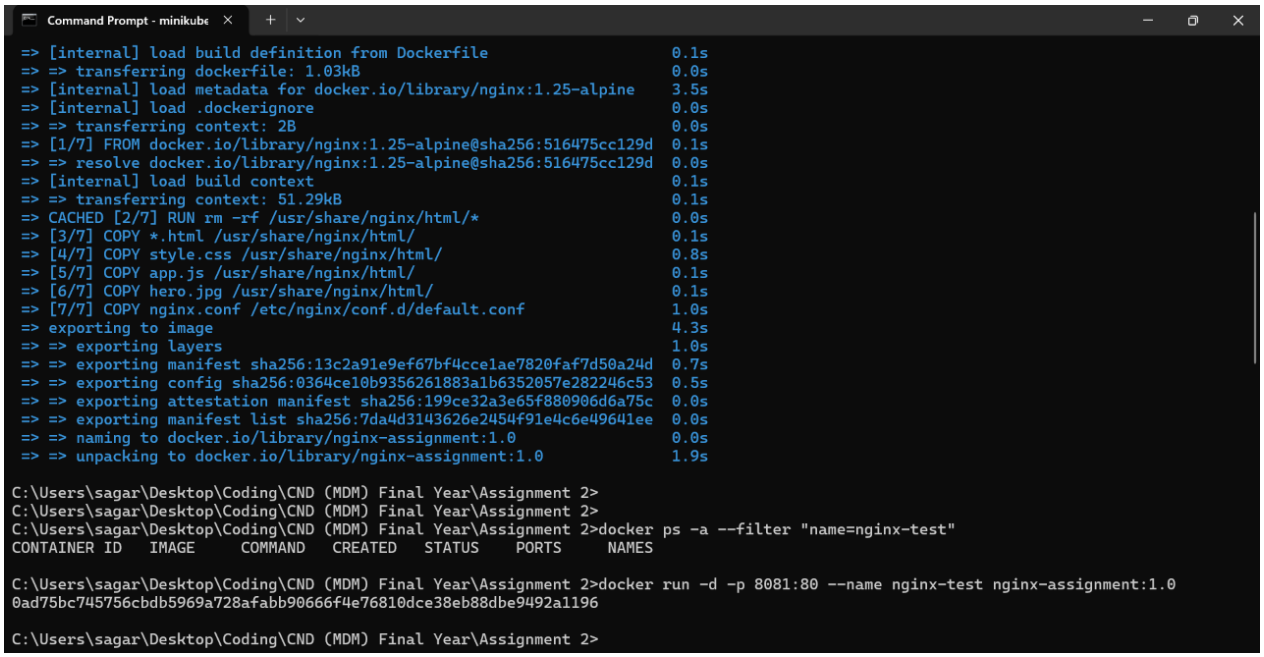
docker build -t nginx-assignment:1.0 .

Verify the image:

docker images nginx-assignment

The image should appear with the repository name nginx-assignment and tag 1.0.





Container Execution and Local Testing

Before deploying the application to Kubernetes, the Docker image is tested as a standalone container.

Running the Container

The following command maps port 8081 on the host to port 80 inside the container:

docker run -d -p 8081:80 --name nginx-test nginx-assignment:1.0

Verify that the container is running:

docker ps --filter "name=nginx-test"

Inspect the container logs:

docker logs nginx-test

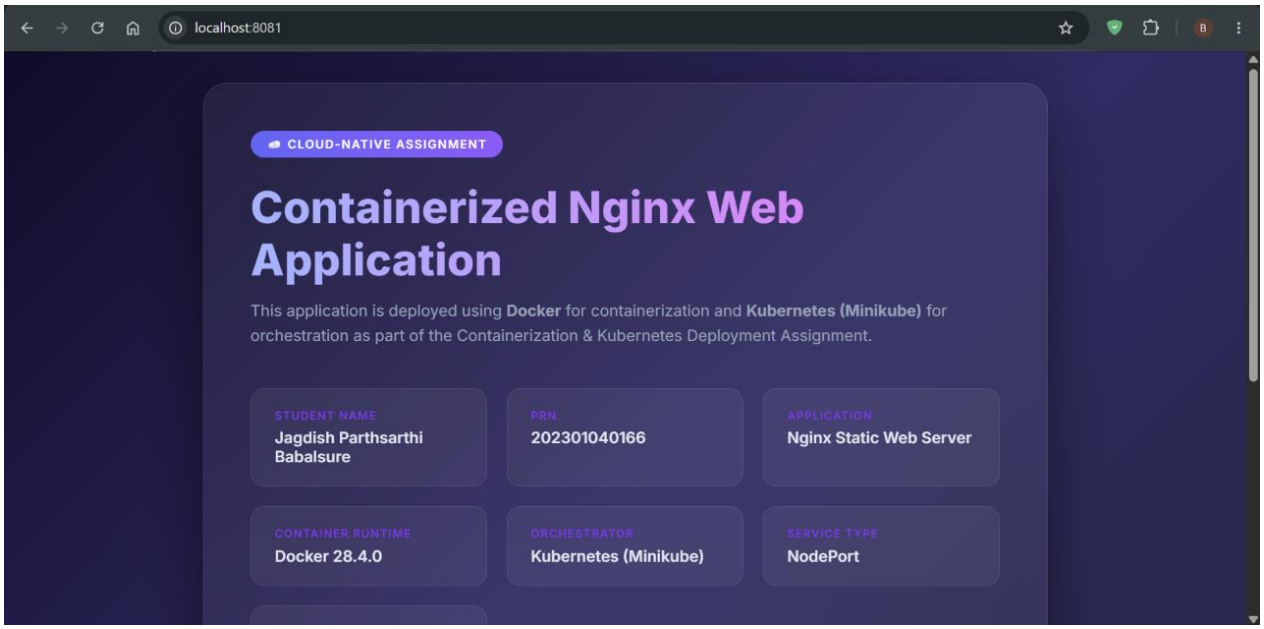
```
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>docker ps -a --filter "name=nginx-test"
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>docker run -d -p 8081:80 --name nginx-test nginx-assignment:1.0
0ad75bc745756cbdb5969a728afabb90666f4e76810dce38eb88dbe9492a1196
```

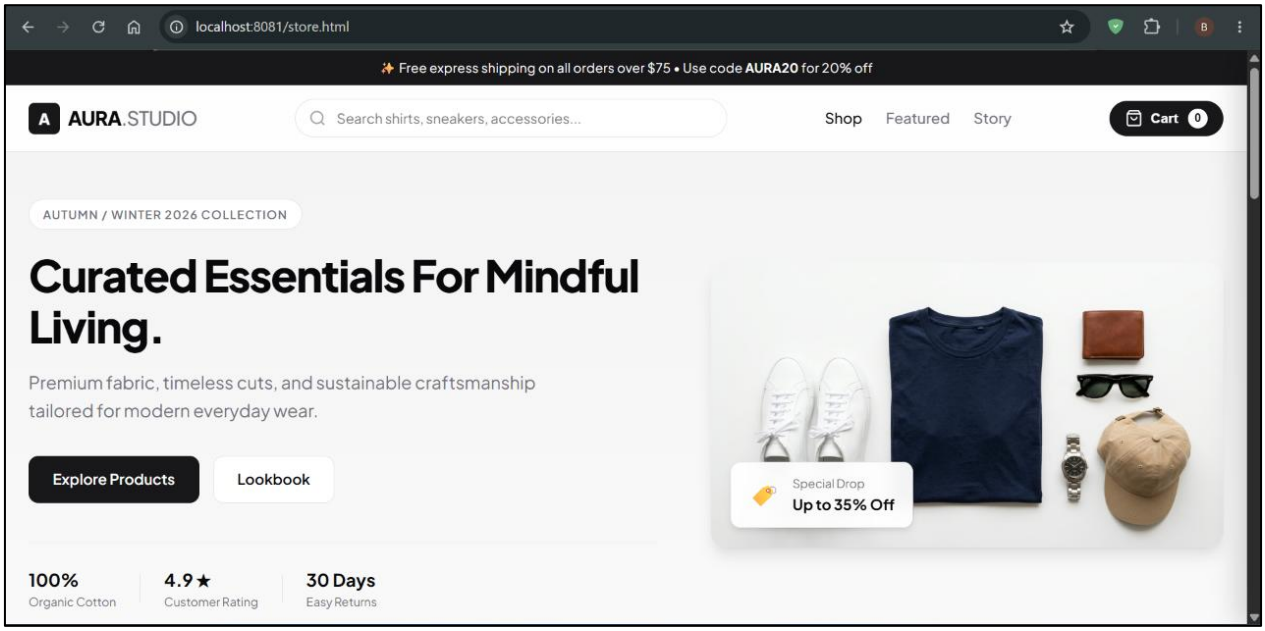
Testing the Application

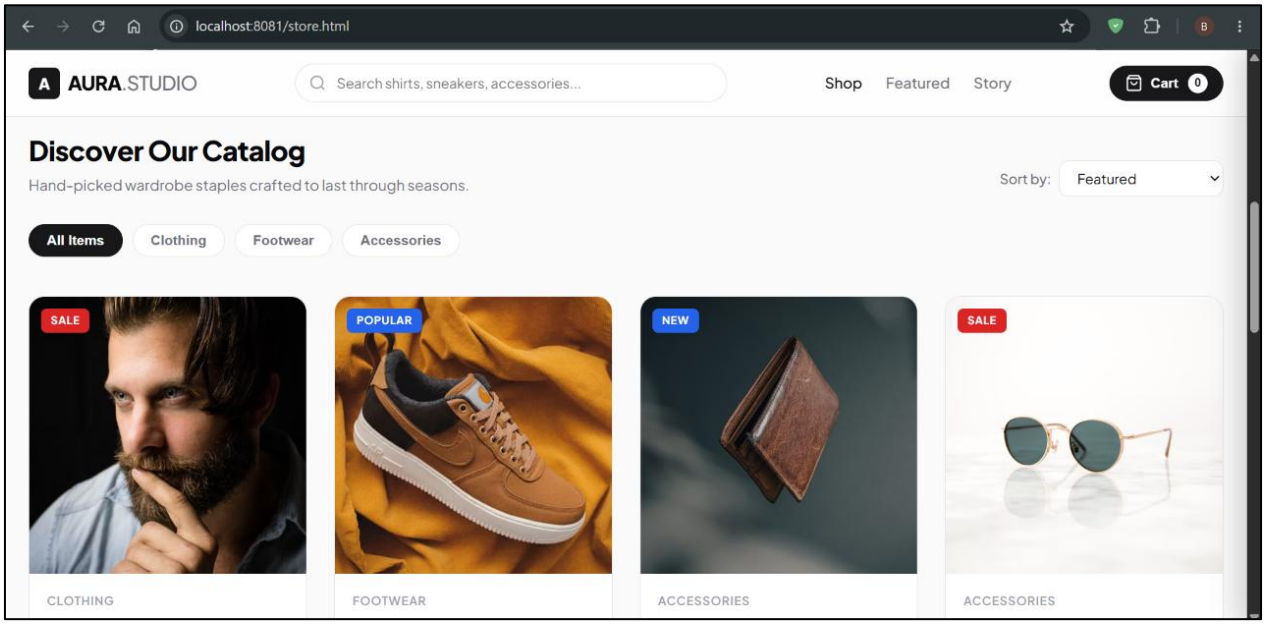
Open the following addresses in a web browser:

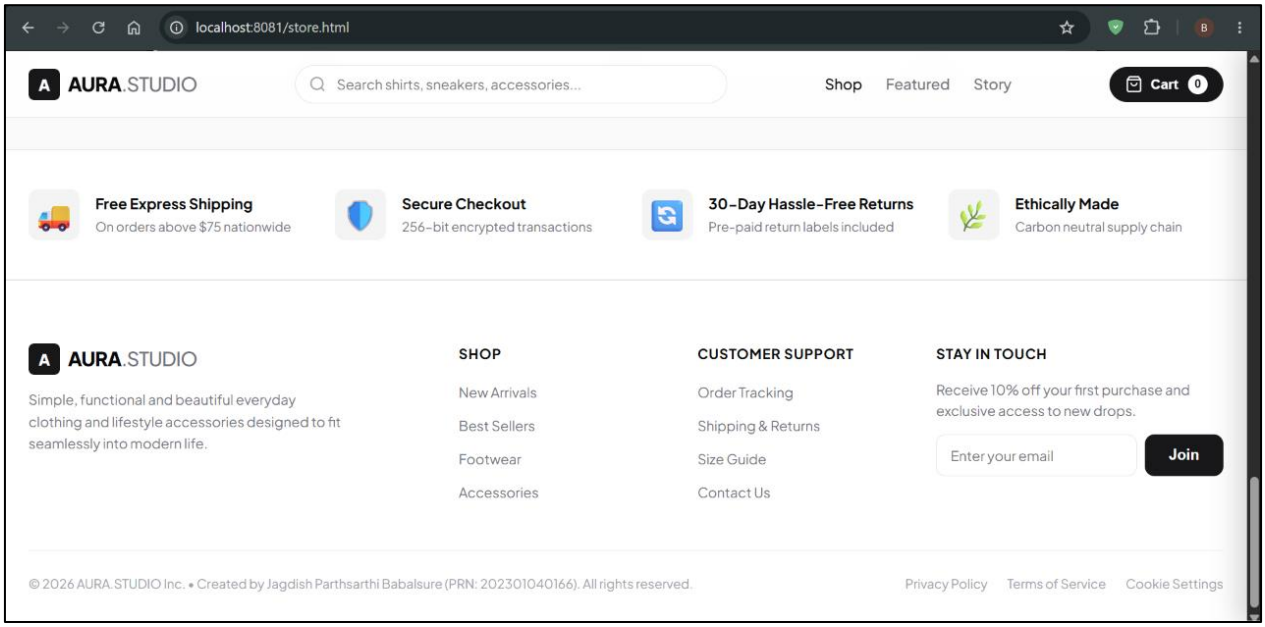
- Dashboard: http://localhost:8081/



- Storefront: http://localhost:8081/store.html


localhost:8081/store.html
✨ Free express shipping on all orders over $75 • Use code AURA20 for 20% off
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
AUTUMN / WINTER 2026 COLLECTION
Curated Essentials For Mindful Living.
Premium fabric, timeless cuts, and sustainable craftsmanship tailored for modern everyday wear.
Explore Products
Lookbook
Special Drop
Up to 35% Off
100%
Organic Cotton
4.9★
Customer Rating
30 Days
Easy Returns


localhost:8081/store.html
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
Discover Our Catalog
Hand-picked wardrobe staples crafted to last through seasons.
Sort by:
Featured
All Items
Clothing
Footwear
Accessories
SALE
POPULAR
NEW
SALE
CLOTHING
FOOTWEAR
ACCESSORIES
ACCESSORIES


localhost:8081/store.html
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
Free Express Shipping
On orders above $75 nationwide
Secure Checkout
256-bit encrypted transactions
30-Day Hassle-Free Returns
Pre-paid return labels included
Ethically Made
Carbon neutral supply chain
AURA.STUDIO
Simple, functional and beautiful everyday clothing and lifestyle accessories designed to fit seamlessly into modern life.
SHOP
New Arrivals
Best Sellers
Footwear
Accessories
CUSTOMER SUPPORT
Order Tracking
Shipping & Returns
Size Guide
Contact Us
STAY IN TOUCH
Receive 10% off your first purchase and exclusive access to new drops.
Enter your email
Join
© 2026 AURA.STUDIO Inc. • Created by Jagdish Parthsarthi Babalsure (PRN: 202301040166). All rights reserved.
Privacy Policy
Terms of Service
Cookie Settings

Kubernetes Cluster Setup

Minikube is used to run a local Kubernetes cluster. Docker Desktop provides the container runtime for the cluster.

Starting Minikube

Ensure Docker Desktop is running and execute:

minikube start --driver=docker

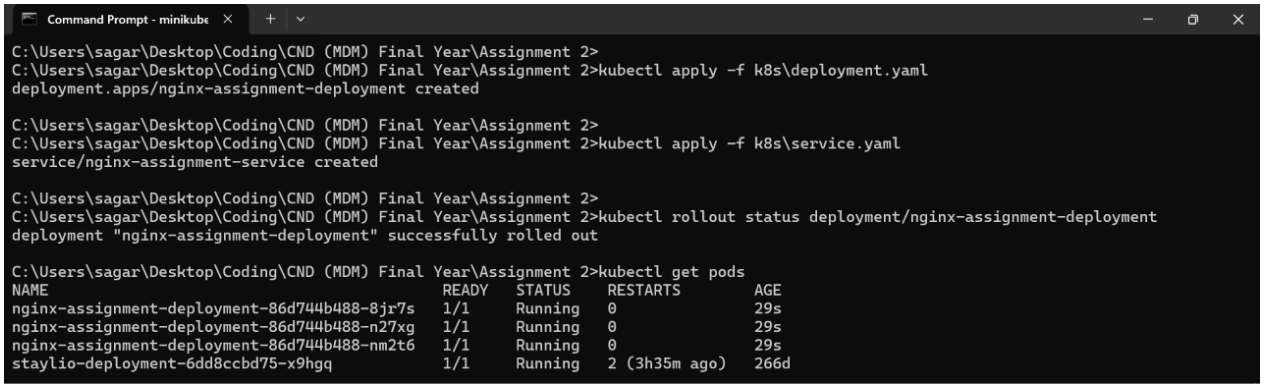
Check the cluster status:

minikube status

Verify that Kubernetes recognizes the node:

kubectl get nodes

The node should show a Ready status before deploying the application.



Loading the Docker Image into Minikube

The Kubernetes cluster must be able to access the application image. For this project, the locally built image is loaded into Minikube using:

minikube image load nginx-assignment:1.0

Verify that the image is available:

minikube image ls

Check the output for the nginx-assignment:1.0 image.

The Deployment manifest uses the locally available image, avoiding the need to retrieve the assignment image from a remote registry.



Kubernetes Deployment Configuration

A Kubernetes Deployment manages the application Pods and maintains the desired replica count.

The project Deployment is configured for three replicas. It also specifies a rolling-update strategy, resource requests and limits, and HTTP liveness and readiness probes.

k8s/deployment.yaml

Use the following configuration if it matches your final manifest:

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: nginx-assignment-deployment
```

```
  namespace: default
  labels:
    app: nginx-assignment
    version: "1.0"
    tier: frontend

spec:
  replicas: 3

  selector:
    matchLabels:
      app: nginx-assignment

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0

  template:
    metadata:
      labels:
        app: nginx-assignment
        version: "1.0"

    spec:
```

```
containers:
  - name: nginx-app
    image: nginx-assignment:1.0
    imagePullPolicy: Never

    ports:
      - containerPort: 80
        protocol: TCP

    resources:
      requests:
        memory: "64Mi"
        cpu: "50m"
      limits:
        memory: "128Mi"
        cpu: "200m"

    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 15
      failureThreshold: 3

    readinessProbe:
```

```
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
        failureThreshold: 3

    restartPolicy: Always
```

Applying the Deployment

Execute:

kubectl apply -f k8s/deployment.yaml

Wait for the rollout to complete:

kubectl rollout status deployment/nginx-assignment-deployment

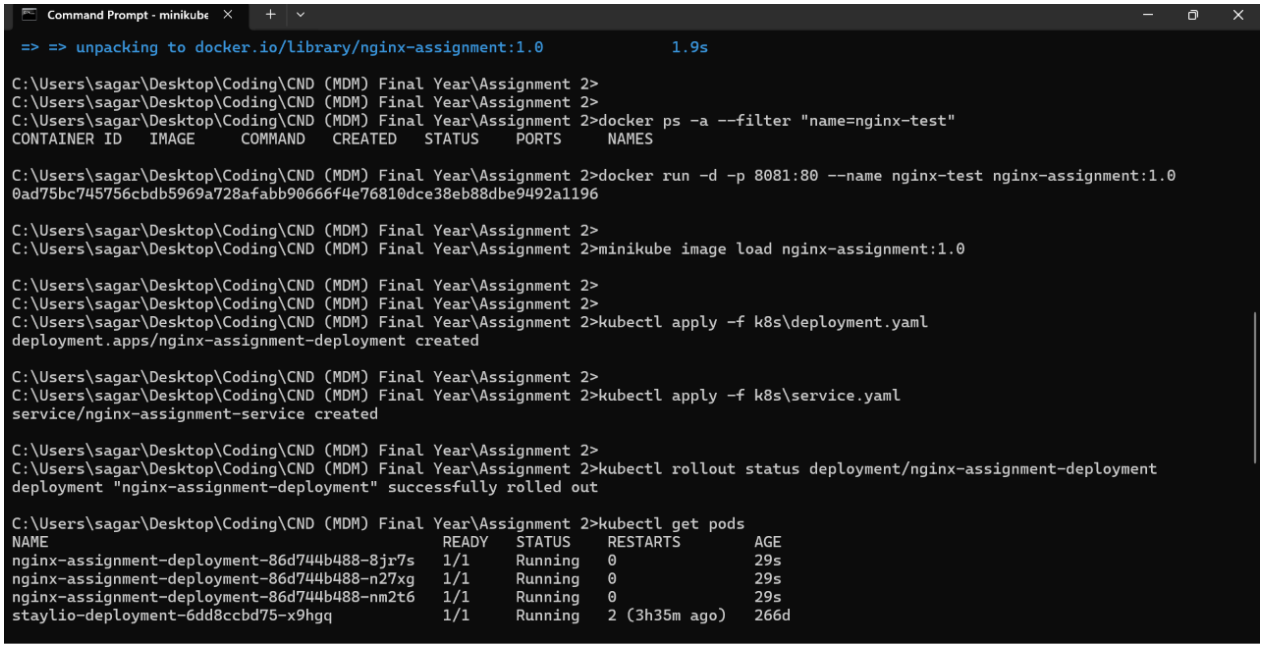
Check the Deployment:

kubectl get deployments

Check the Pods:

kubectl get pods -l app=nginx-assignment -o wide

The expected result is three Pods in the Running state, with each Pod showing 1/1 ready.



Kubernetes Service Configuration

A Kubernetes Service provides a stable access point for the application Pods. The project uses a NodePort Service to expose the application through port 30080 on the Minikube node.

k8s/service.yaml

Use the following configuration if it matches your final manifest:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-assignment-service
  namespace: default
  labels:
    app: nginx-assignment
    tier: frontend
```

```
spec:
  type: NodePort

  selector:
    app: nginx-assignment

  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

Applying and Verifying the Service

Apply the Service configuration:

kubectl apply -f k8s/service.yaml

Verify the Service:

kubectl get services

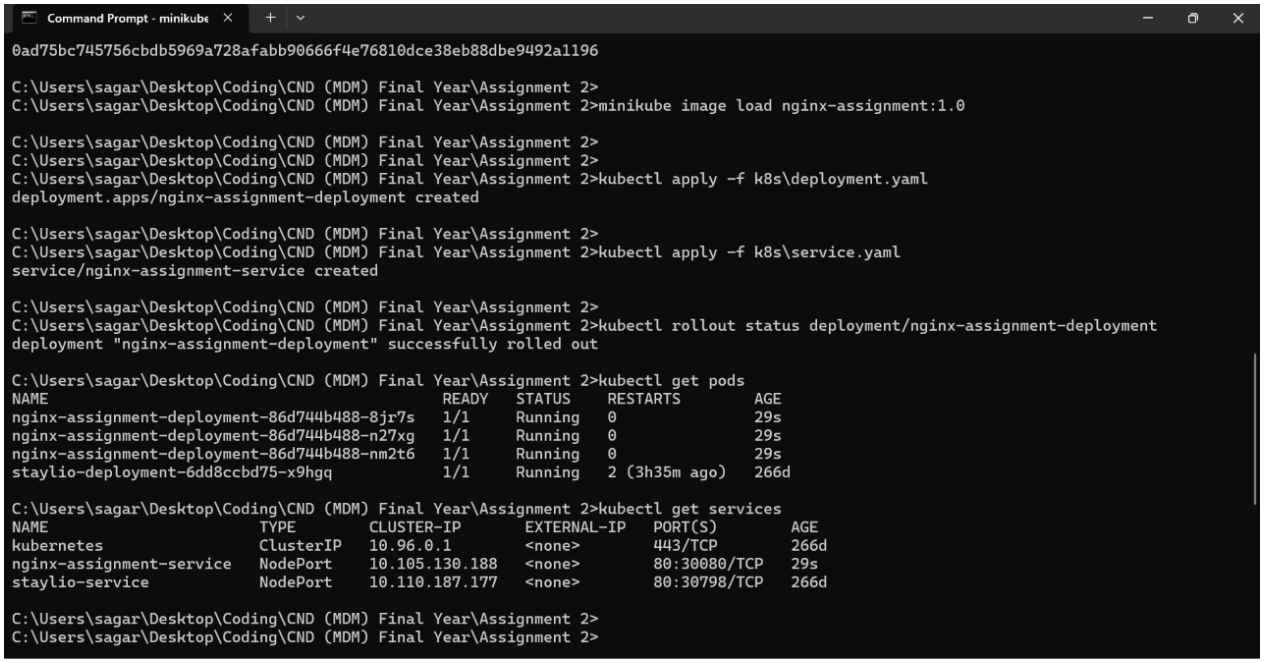
Inspect its configuration:

kubectl describe service nginx-assignment-service

Check its endpoints:

kubectl get endpoints nginx-assignment-service

The Service selector should match the Pod labels defined in the Deployment. Its endpoints should correspond to the application Pods when they are ready.


Command Prompt - minikube
0ad75bc745756cbdb5969a728afabb90666f4e76810dce38eb88dbe9492a1196
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>minikube image load nginx-assignment:1.0
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl apply -f k8s\deployment.yaml
deployment.apps/nginx-assignment-deployment created
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl apply -f k8s\service.yaml
service/nginx-assignment-service created
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl rollout status deployment/nginx-assignment-deployment
deployment "nginx-assignment-deployment" successfully rolled out
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx-assignment-deployment-86d744b488-8jr7s 1/1 Running 0 29s
nginx-assignment-deployment-86d744b488-n27xg 1/1 Running 0 29s
nginx-assignment-deployment-86d744b488-nm2t6 1/1 Running 0 29s
staylio-deployment-6dd8ccbd75-x9hgq 1/1 Running 2 (3h35m ago) 266d
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl get services
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 266d
nginx-assignment-service NodePort 10.105.130.188 <none> 80:30080/TCP 29s
staylio-service NodePort 10.110.187.177 <none> 80:30798/TCP 266d
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>

Accessing the Application Through Kubernetes

The application can be accessed through the Minikube Service command:

minikube service nginx-assignment-service

This command provides a URL for accessing the application through the Kubernetes Service. Keep the associated terminal process running if Minikube uses a local tunnel or forwarding process.

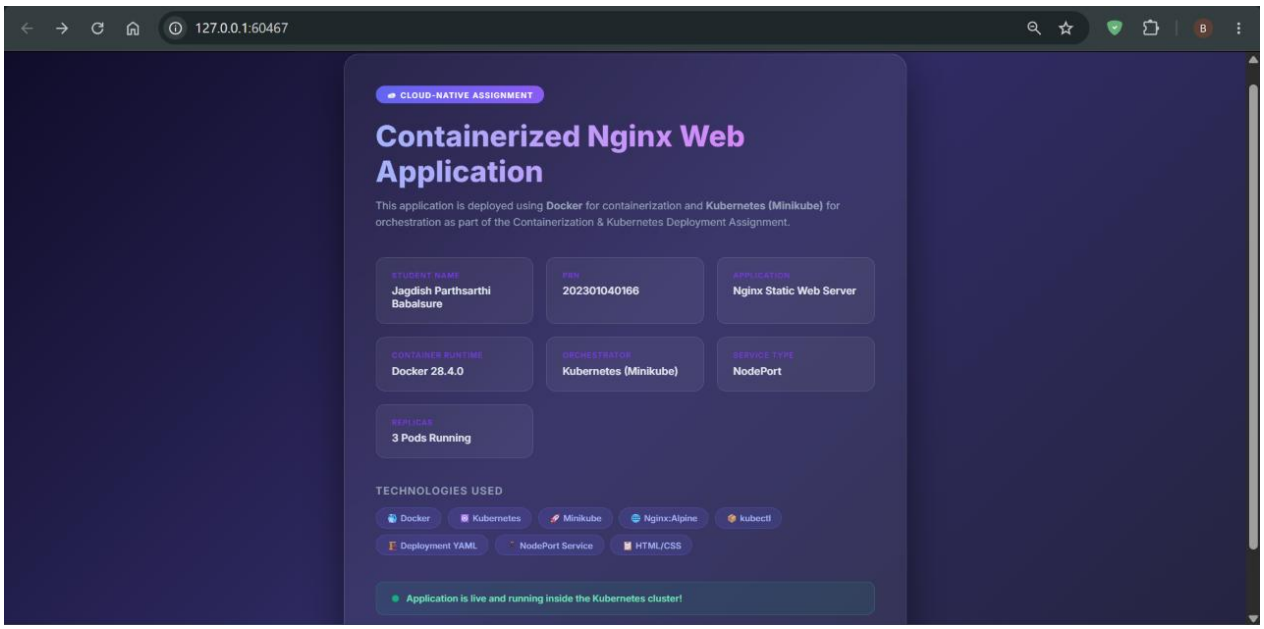
The landing page is available at the root URL. To access the storefront directly, append /store.html to the same URL.

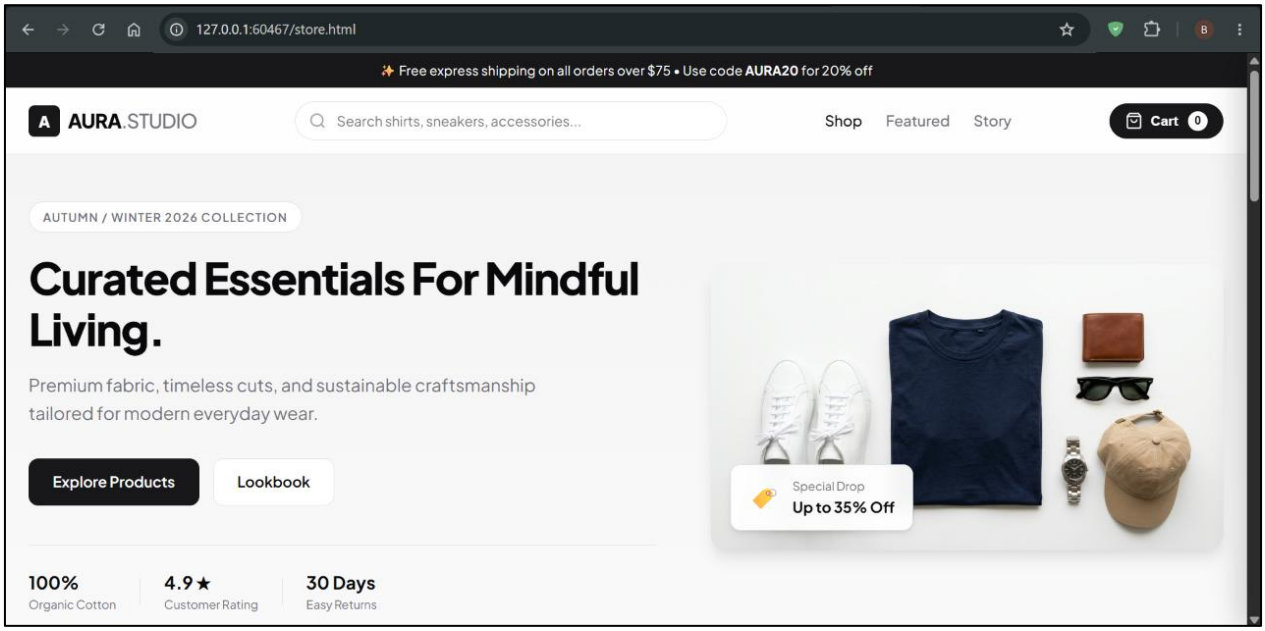
For example, if Minikube provides a URL similar to http://127.0.0.1:60467/, use:

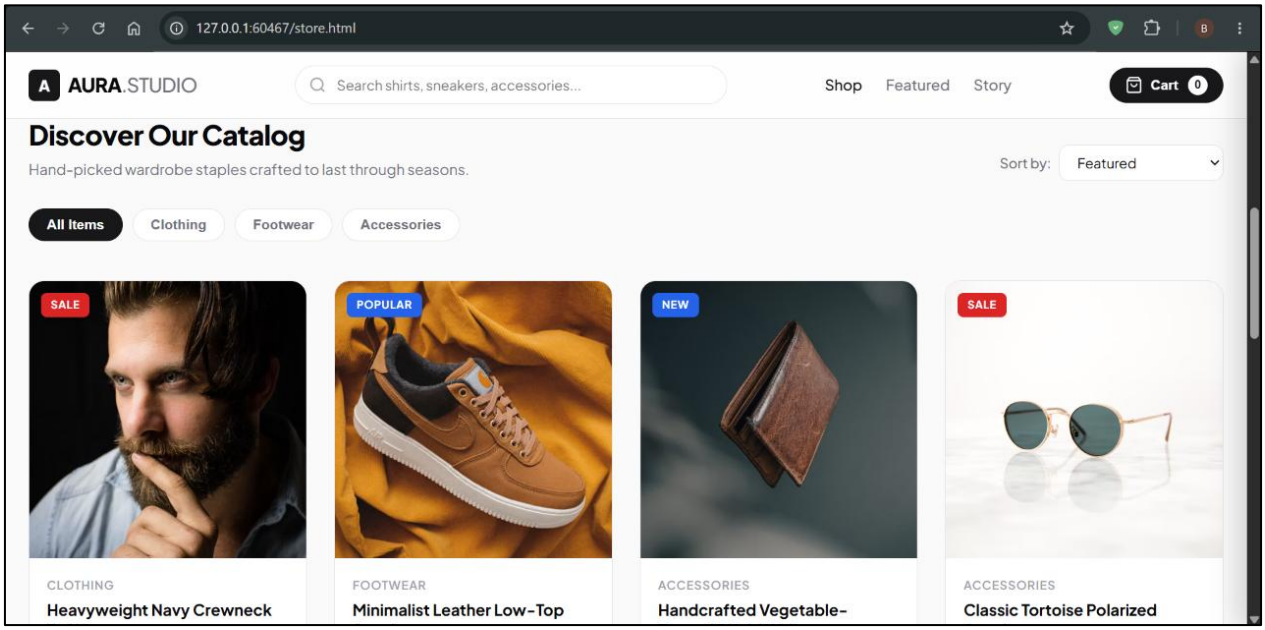
- Dashboard: http://127.0.0.1:60467/
- Storefront: http://127.0.0.1:60467/store.html

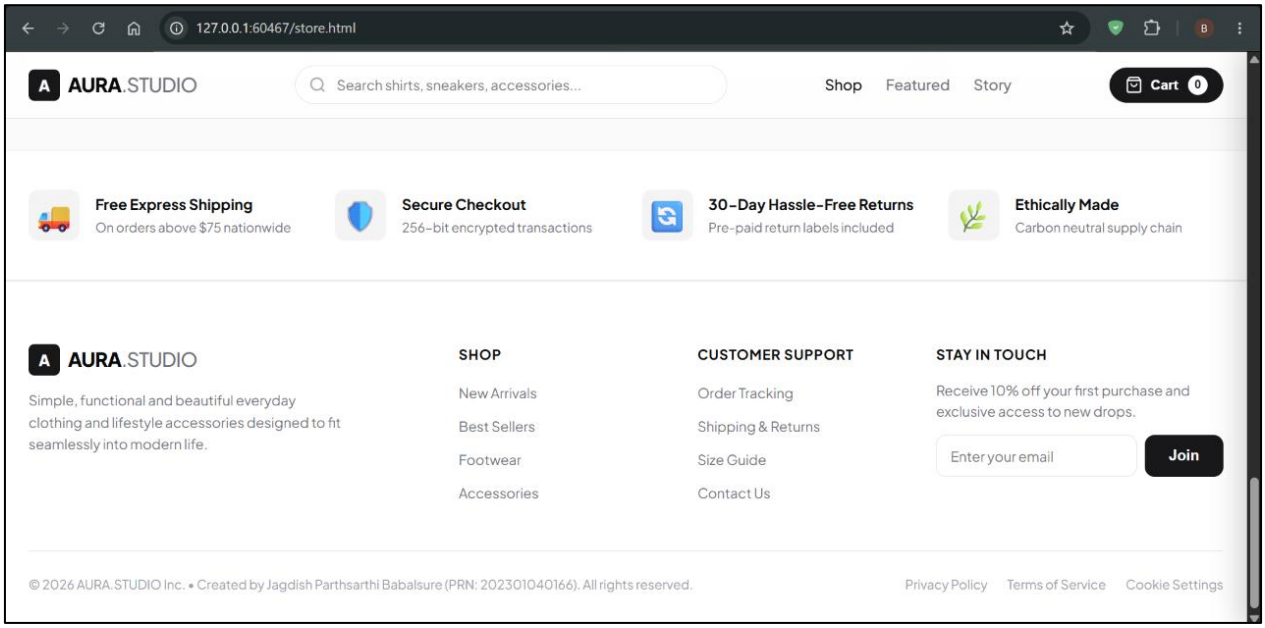
The port may change when the command is run again; use the URL actually displayed by Minikube.

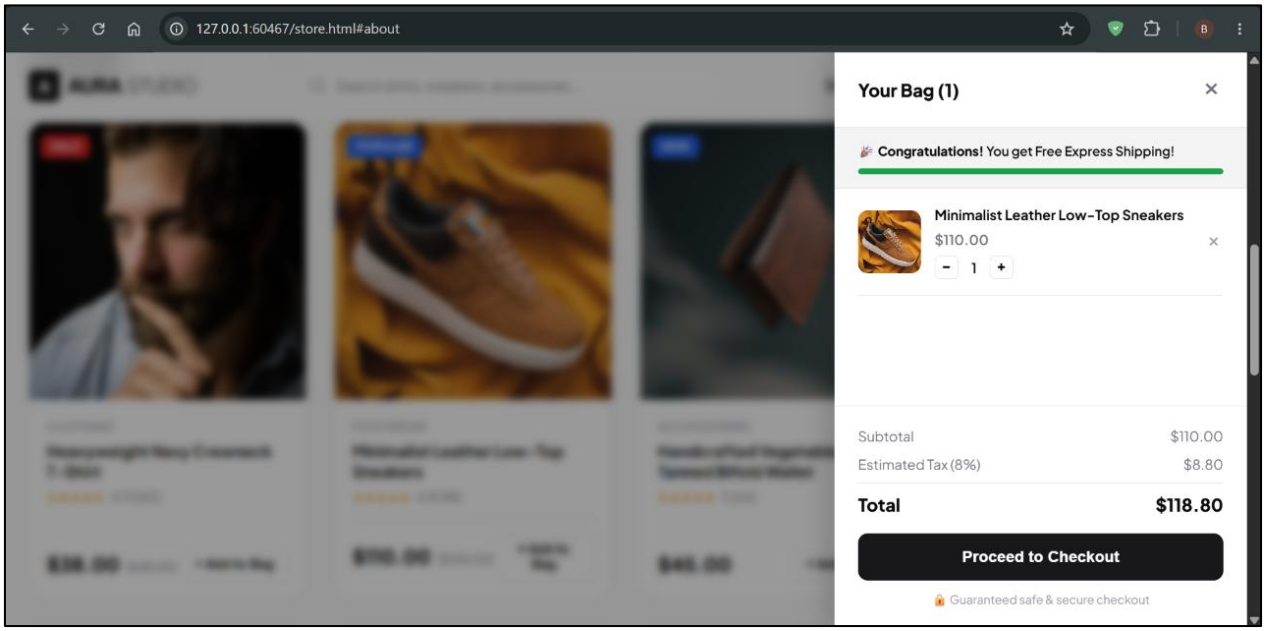
Application Running Through Kubernetes

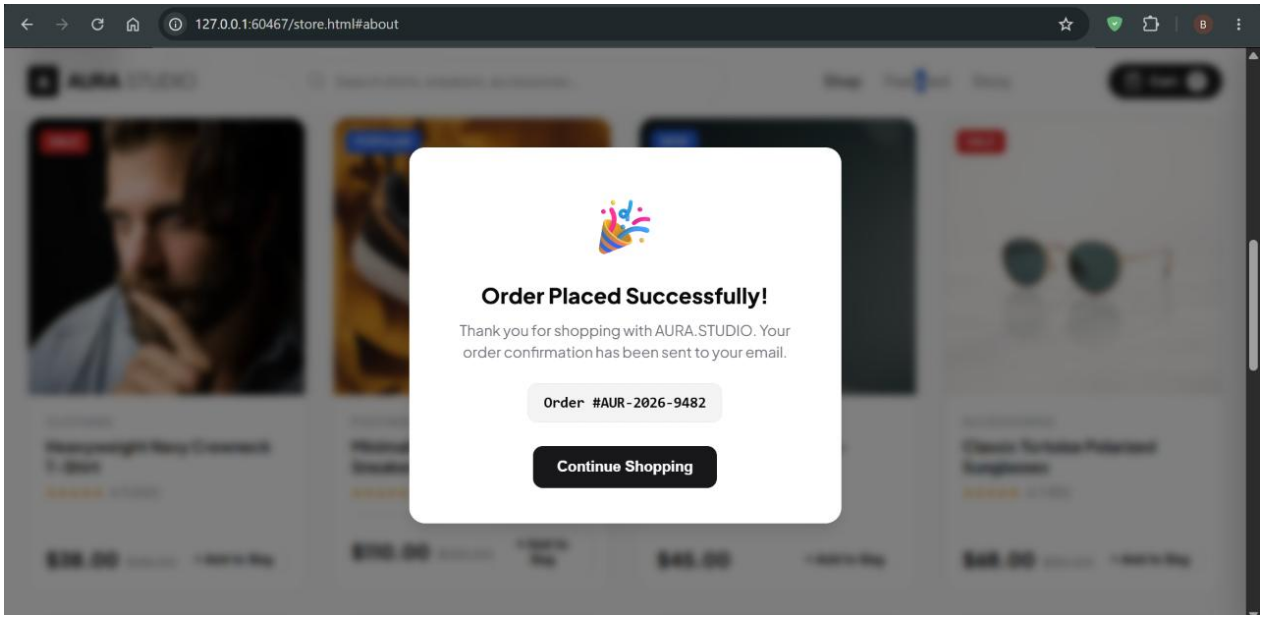


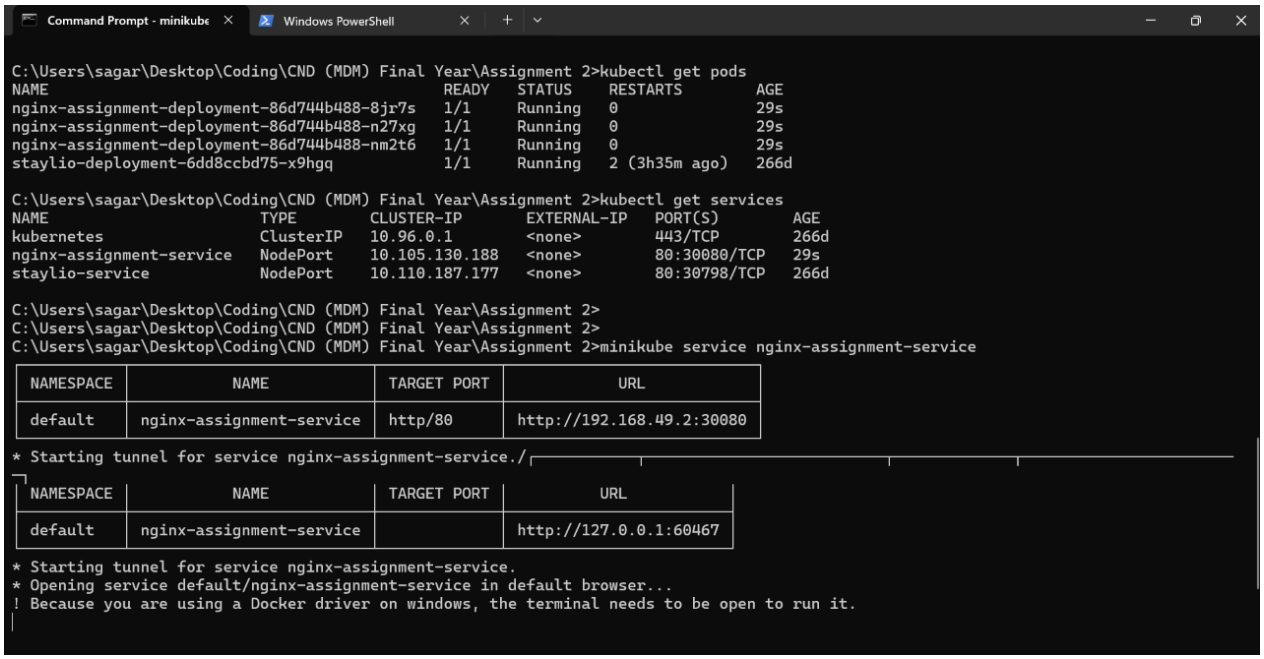

127.0.0.1:60467/store.html
✨ Free express shipping on all orders over $75 • Use code AURA20 for 20% off
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
AUTUMN / WINTER 2026 COLLECTION
Curated Essentials For Mindful Living.
Premium fabric, timeless cuts, and sustainable craftsmanship tailored for modern everyday wear.
Explore Products
Lookbook
Special Drop
Up to 35% Off
100%
Organic Cotton
4.9★
Customer Rating
30 Days
Easy Returns


127.0.0.1:60467/store.html
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
Discover Our Catalog
Hand-picked wardrobe staples crafted to last through seasons.
Sort by:
Featured
All Items
Clothing
Footwear
Accessories
SALE
POPULAR
NEW
SALE
CLOTHING
Heavyweight Navy Crewneck
FOOTWEAR
Minimalist Leather Low-Top
ACCESSORIES
Handcrafted Vegetable-
ACCESSORIES
Classic Tortoise Polarized


127.0.0.1:60467/store.html
AURA.STUDIO
Search shirts, sneakers, accessories...
Shop
Featured
Story
Cart 0
Free Express Shipping
On orders above $75 nationwide
Secure Checkout
256-bit encrypted transactions
30-Day Hassle-Free Returns
Pre-paid return labels included
Ethically Made
Carbon neutral supply chain
AURA.STUDIO
Simple, functional and beautiful everyday clothing and lifestyle accessories designed to fit seamlessly into modern life.
SHOP
New Arrivals
Best Sellers
Footwear
Accessories
CUSTOMER SUPPORT
Order Tracking
Shipping & Returns
Size Guide
Contact Us
STAY IN TOUCH
Receive 10% off your first purchase and exclusive access to new drops.
Enter your email
Join
© 2026 AURA.STUDIO Inc. • Created by Jagdish Parthsarthi Babalsure (PRN: 202301040166). All rights reserved.
Privacy Policy
Terms of Service
Cookie Settings


127.0.0.1:60467/store.html#about
Your Bag (1)
Congratulations! You get Free Express Shipping!
Minimalist Leather Low-Top Sneakers
$110.00
1
Subtotal
$110.00
Estimated Tax (8%)
$8.80
Total
$118.80
Proceed to Checkout
Guaranteed safe & secure checkout


127.0.0.1:60467/store.html#about
Order Placed Successfully!
Thank you for shopping with AURA.STUDIO. Your order confirmation has been sent to your email.
Order #AUR-2026-9482
Continue Shopping


Command Prompt - minikube
Windows PowerShell
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl get pods
NAME READY STATUS RESTARTS AGE
nginx-assignment-deployment-86d744b488-8jr7s 1/1 Running 0 29s
nginx-assignment-deployment-86d744b488-n27xg 1/1 Running 0 29s
nginx-assignment-deployment-86d744b488-nm2t6 1/1 Running 0 29s
staylio-deployment-6dd8ccbd75-x9hgq 1/1 Running 2 (3h35m ago) 266d
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>kubectl get services
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 266d
nginx-assignment-service NodePort 10.105.130.188 <none> 80:30080/TCP 29s
staylio-service NodePort 10.110.187.177 <none> 80:30798/TCP 266d
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>
C:\Users\sagar\Desktop\Coding\CND (MDM) Final Year\Assignment 2>minikube service nginx-assignment-service
NAMESPACE NAME TARGET PORT URL
default nginx-assignment-service http/80 http://192.168.49.2:30080
* Starting tunnel for service nginx-assignment-service.
NAMESPACE NAME TARGET PORT URL
default nginx-assignment-service http://127.0.0.1:60467
* Starting tunnel for service nginx-assignment-service.
* Opening service default/nginx-assignment-service in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.

Verification and Monitoring

Kubernetes provides commands to inspect the deployment, Pods, Service, and application logs.

Viewing Kubernetes Resources

kubectl get all -l app=nginx-assignment

Inspecting Pod Logs

kubectl logs -l app=nginx-assignment --tail=20

Inspecting Deployment Details

kubectl describe deployment nginx-assignment-deployment

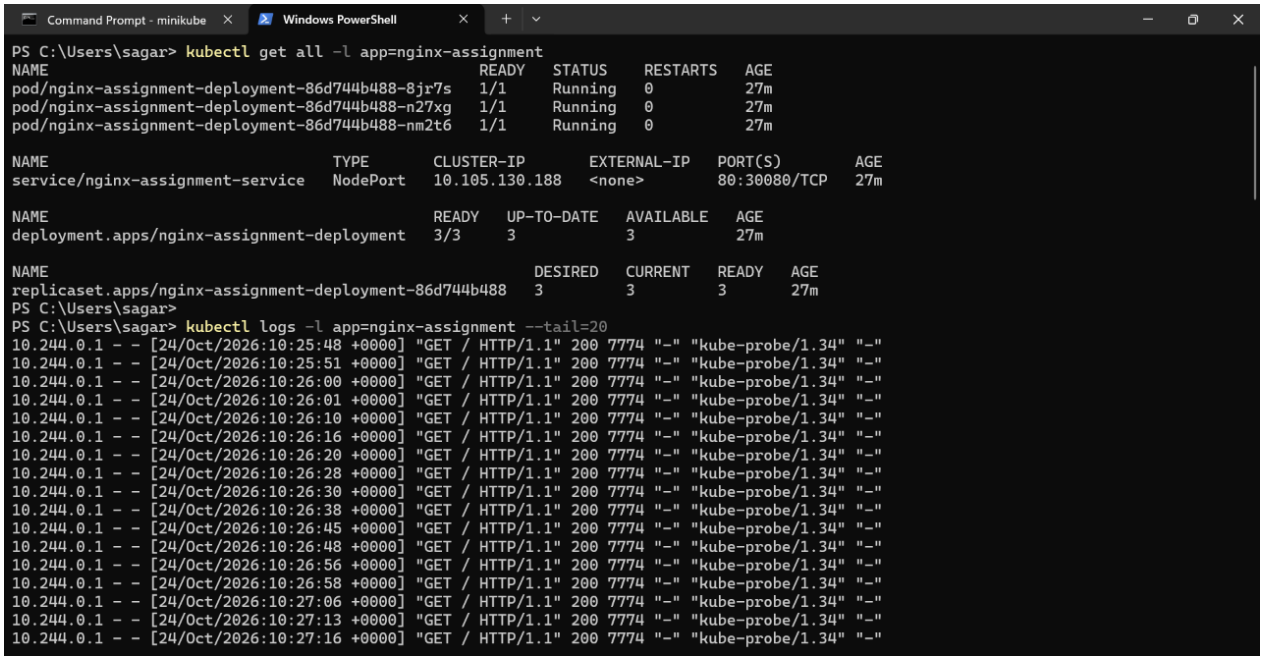
Checking Pod Status

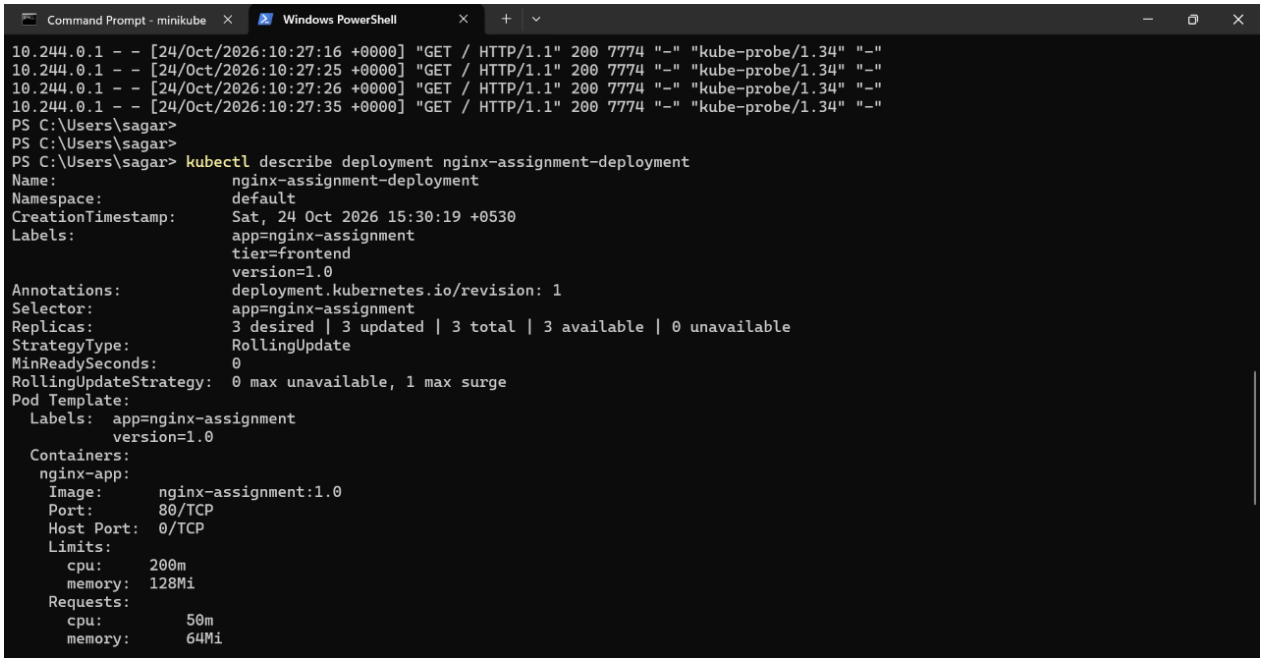
kubectl get pods -o wide

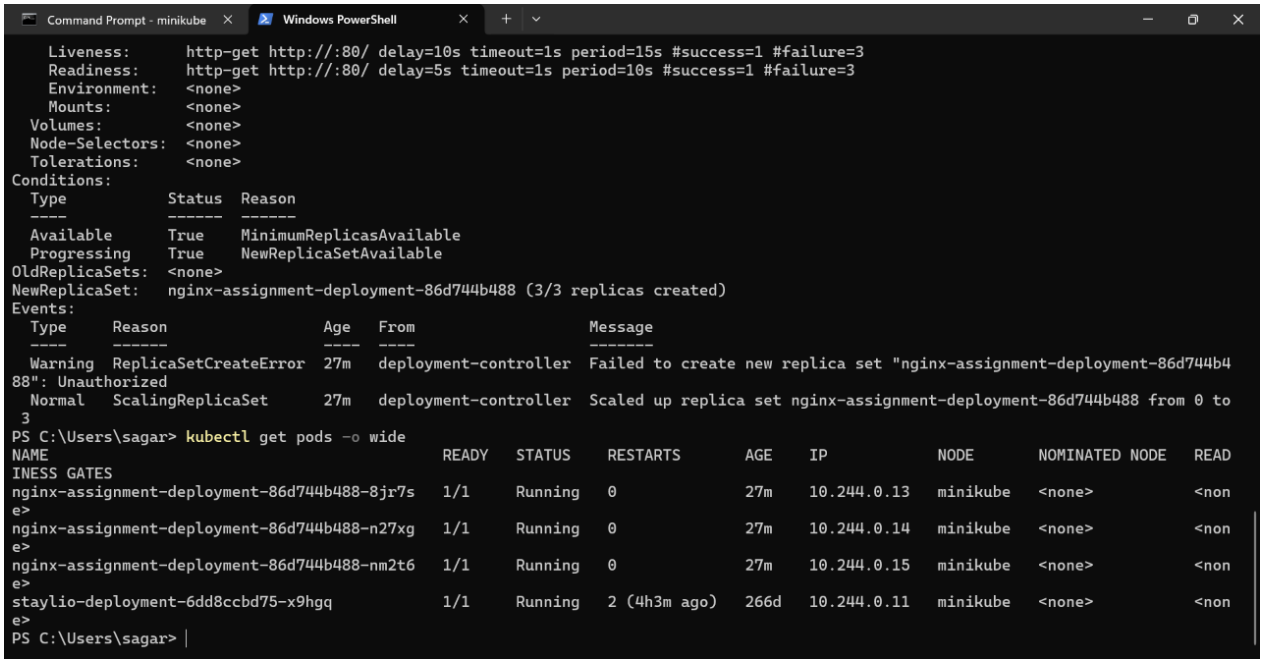
These commands help verify the current state of the deployment, inspect configuration details, and troubleshoot problems.

The liveness probe checks whether the configured HTTP endpoint is responding. The readiness probe checks whether the Pod should receive Service traffic. Resource requests and limits specify the CPU and memory resources requested and permitted for each container.

Kubernetes Resource Verification






Command Prompt - minikube
Windows PowerShell
Liveness: http-get http://:80/ delay=10s timeout=1s period=15s #success=1 #failure=3
Readiness: http-get http://:80/ delay=5s timeout=1s period=10s #success=1 #failure=3
Environment: <none>
Mounts: <none>
Volumes: <none>
Node-Selectors: <none>
Tolerations: <none>
Conditions:
Type Status Reason
Available True MinimumReplicasAvailable
Progressing True NewReplicaSetAvailable
OldReplicaSets: <none>
NewReplicaSet: nginx-assignment-deployment-86d744b488 (3/3 replicas created)
Events:
Type Reason Age From Message
Warning ReplicaSetCreateError 27m deployment-controller Failed to create new replica set "nginx-assignment-deployment-86d744b488": Unauthorized
Normal ScalingReplicaSet 27m deployment-controller Scaled up replica set nginx-assignment-deployment-86d744b488 from 0 to 3
PS C:\Users\sagar> kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
nginx-assignment-deployment-86d744b488-8jr7s 1/1 Running 0 27m 10.244.0.13 minikube <none> <none>
nginx-assignment-deployment-86d744b488-n27xg 1/1 Running 0 27m 10.244.0.14 minikube <none> <none>
nginx-assignment-deployment-86d744b488-nm2t6 1/1 Running 0 27m 10.244.0.15 minikube <none> <none>
staylio-deployment-6dd8ccbd75-x9hgq 1/1 Running 2 (4h3m ago) 266d 10.244.0.11 minikube <none> <none>
PS C:\Users\sagar>

GitHub Repository

The project source code and Kubernetes configuration are maintained in the following GitHub repository:

Repository URL:
https://github.com/jagdishbabalsure/CLOUD-NATIVE-DEVOPS-MDM/tree/main/Assignment%202

The repository includes the Dockerfile, HTML pages, stylesheet, JavaScript, Nginx configuration, Kubernetes manifests, deployment script, and README file.

Version control makes it possible to maintain the source code and deployment configuration and reproduce the setup on another compatible development environment.

Conclusion

This assignment demonstrates the containerization and Kubernetes deployment of a static web application using Docker, Nginx, and Minikube.

The application was packaged into a Docker image named nginx-assignment:1.0 and tested in a standalone container. The image was then made available to Minikube and deployed using a Kubernetes Deployment configured with three replicas.

A NodePort Service was configured to expose the application, and Kubernetes commands were used to inspect the node, deployment, Pods, Service, and application logs. The dashboard and storefront were also tested through the application URL provided by Minikube.

The assignment provided practical experience with Docker image creation, container execution, Kubernetes Deployments and Services, resource configuration, health probes, and application access through a local Kubernetes cluster.

Overall, the project demonstrates the basic workflow of containerizing a web application and deploying it with Kubernetes in a local development environment.